



Final Project Document

X (Twitter)

Advanced Programming - Summer 2026

Course Instructor: Dr. Saeed Reza Kheradpisheh



1. Introduction

1.1 Project Overview

This project involves the development of a desktop social media application inspired by X (formerly Twitter). The application will be built using **Java**, with JavaFX/Swing for the graphical user interface and PostgreSQL or MySQL as the database management system.

The main goal is to create a functional social networking platform that allows users to publish tweets, follow other users, interact with content through likes and replies, and view personalized feeds. This project provides an opportunity to apply **object-oriented programming principles**, **GUI development** with JavaFX or Swing, and **database management skills** in a practical, real-world scenario.

1.2 Educational Objectives

This project provides practical experience in:

- Developing a **client-server application** using Java sockets
- Applying **object-oriented** design principles
- Designing and implementing **PostgreSQL/MySQL** databases
- Using **JDBC/ORM** and parameterized queries for database access
- Building **graphical user interfaces** with JavaFX/Swing
- Creating a JSON-based **communication protocol**
- Implementing multithreading for **concurrent client support**
- Using **Git** and collaborative software development practices
- Producing **technical documentation**, including ERD and architecture diagrams

2. Exploring Twitter/X

Before beginning development, explore Twitter/X and analyze its core features and user workflows. Pay particular attention to user registration and authentication, profile management, following relationships, tweet creation and interactions, personalized feeds, search functionality, notifications, hashtags, trends, and media sharing.

As you explore the platform, document how users interact with each feature and identify the data and functionality required to implement similar behavior in your project. This analysis will serve as the foundation for your system design and development process.

3. Main Tasks (Mandatory Features)

All features in this section must be fully implemented and work correctly in a multi-client environment.

3.1 Client–Server Architecture Requirements

The application must follow a client–server architecture.

- A central server must manage all users, tweets, media, and relationships.
- Multiple clients must be able to connect to the server simultaneously.
- All persistent data must be stored centrally in a PostgreSQL or MySQL database.
- Clients and the server must communicate using a JSON-based protocol.
- The server must support concurrent client connections using multithreading.
- User actions performed on one client must be reflected across the system for all connected users.
- Newly published tweets should be delivered to relevant connected users **in real time** without requiring a manual refresh.
- Real-time updates for likes, follows, notifications, and similar interactions are considered **bonus features**.

3.2 User Management

The system must support complete account and profile management.

- **User Registration** – Create accounts using a unique username, email, and password.
- **User Login** – Authenticate users and create active sessions.
- **User Logout** – Securely terminate user sessions.
- **Password Security** – Validate passwords and store them using secure hashing.
- **Unique Usernames** – Prevent duplicate usernames.
- **Profile Creation** – Set display name, bio, and profile picture.
- **Profile Editing** – Update profile information and appearance.
- **Avatar Upload** – Upload and display profile pictures.
- **User Biography** – Support personal profile descriptions.
- **Profile Viewing** – View both personal and other users' profiles.

3.3 Tweets

Tweets are the main content shared on the platform.

- **Create Tweets** – Publish tweets and deliver them to followers.
- **View Tweets** – Display content, author, timestamp, and media.

- **Delete Tweets** – Remove a user's own tweets.
- **Timestamp Tracking** – Record and display posting time.
- **Tweet Details** – View complete tweet pages and replies.
- **User Tweet History** – Display all tweets from a specific user.

3.4 Tweet Interactions

Users should be able to interact with tweets in multiple ways.

- **Replies** – Respond to tweets.
- **Threaded Conversations** – Display reply chains in an organized format.
- **Retweets/Reposts** – Share existing tweets.

3.5 Hashtags

Hashtags help organize and discover content.

- **Automatic Detection** – Identify hashtags within tweet text.
- **Hashtag Search** – Find tweets using a hashtag.

3.6 Media

Tweets may include image content.

- **Image Attachments** – Add images to tweets.
- **Multiple Images** – Support several images per tweet.
- **Image Previews** – Show thumbnails within the timeline.

3.7 Profile Enhancements

Profiles should provide additional personalization and statistics.

- **Banner Images** – Customize profile headers.
- **Profile Statistics** – Display tweet, follower, and following counts.

3.8 Social Features

The platform must support user-to-user connections.

- **Follow Users** – Subscribe to another user's content.
- **Unfollow Users** – Remove a following relationship.
- **Followers List** – View users who follow an account.
- **Following List** – View users being followed.

- **Follower Count** – Display total followers.
- **Following Count** – Display total following users.

3.9 Likes

Users can express appreciation for tweets.

- **Like Tweets** – Add a like to a tweet.
- **Unlike Tweets** – Remove a previously added like.
- **Like Counts** – Display the number of likes on each tweet.

3.10 Feed

The feed is the primary content consumption area.

- **Personalized Timeline** - Display the user's tweets and tweets from followed users, ordered chronologically.
- **Real-Time Tweet Delivery** – Newly published tweets from followed users should appear automatically in the feed of connected users without requiring manual refresh.

3.11 Search

Users must be able to find content and accounts efficiently.

- **User Search** – Search by username or display name.
- **Profile Navigation** – Open profiles directly from search results.
- **Tweet Search** – Find tweets using keywords.
- **Hashtag Search** – Search for hashtag-related content.

4. Bonus Features

- Direct Messaging – Enable private communication between users.
- Quote Tweets – Repost tweets with additional commentary.
- Tweet Editing with Edit History – Let users edit tweets while preserving previous versions.
- User Blocking – Prevent specific users from viewing or interacting with an account.
- User Muting – Hide posts and notifications from selected users without unfollowing them.
- Verification Badge – Indicate verified accounts.
- Notifications
 - Real-Time Notification Delivery
 - New Follower Notifications
 - Like & Tweet Notifications
- Real-Time UI Updates – Automatically update feeds, notifications, and interactions without requiring refreshes.
- Suggested Users to Follow – Recommend accounts based on activity or mutual connections.
- Suggested Hashtags – Provide hashtag recommendations while creating posts.
- Hashtag Following – Subscribe to specific hashtags.
- Trending Hashtags – Display popular hashtags based on recent activity.
- Poll Creation and Voting – Enable users to create polls and vote on them.
- Pinned Tweets – Allow users to highlight a selected tweet at the top of their profile.
- Scheduled Tweets – Allow users to schedule posts for future publication.
- Video Attachments – Allow users to upload and share videos with their posts.
- GIF Support – Enable posting and viewing animated GIFs.
- Image Full-Size Viewing – Open images in their original size.
- Profile-Specific Search – Search within a user's posts.
- Character Limit Enforcement – Restrict posts to a maximum number of characters, similar to Twitter/X.
- Dark Mode – Provide a dark-themed interface for improved readability and comfort.
- Theme Customization – Allow users to personalize colors and visual appearance.
- Responsive UI – Adapt the interface to different window sizes and resolutions.
- **Creativity Bonus** – Impress us with an original, unexpected, or delightfully unique feature that goes beyond the standard checklist. Surprise us with something memorable.

5. README Requirements

A **README** file is a guide that gives users a description of the project you have worked on, with guidelines on how to use a project. It is the first file a person will see when they encounter your project, so it should be fairly brief but detailed. A well-written README can also help you focus on what helps you focus on what your project needs to deliver and how. Your project's README should be able to answer the what, how, and most importantly the why of the project.

Your README should include the following sections:

- **Project Title:** State the project's name.
- **Table of Contents:** A table of contents can help users navigate to specific sections.
- **Descriptions:** Provide an overview of what the project does and its purpose.
- **Usage:** Explain how to use the project. Include code examples where you see fit.
- **Demo/GIFs/Images:** Provide visuals to showcase the project in action.
- **Credits:** Acknowledge and attribute any contributors, libraries, dependencies, or resources used in the project.
- **Changelog:** Document changes made in each version of the project.
- **Contact Information:** Offer ways for users to reach out for help or support.

6. Project Report

In addition to the README, each team must submit a **Report.md** file in the root of the repository. The report should describe the design, architecture, implementation, and development process of the project. Its purpose is to explain how the system was built and justify the technical decisions made by the team.

The report should include the following sections:

System Architecture

Describe the overall architecture of the application, including:

- Client and server components
- Communication flow between components
- Major modules and responsibilities
- Architectural patterns used (e.g., layered architecture, MVC)

Include architecture diagrams where appropriate.

Database Design

Describe the database design and implementation.

Include:

- Entity Relationship Diagram (ERD)
- Description of major entities and relationships

Object-Oriented Design

Explain how object-oriented principles were applied in the project.

Include:

- Class diagrams (UML) if created
- Key classes and their responsibilities
- Inheritance, composition, and other design concepts used
- Design patterns, if any

AI Usage Disclosure

If any AI tools were used during the project, the team must disclose their usage.

Include:

- AI tool(s) and model(s) used
- Purpose of usage (e.g., code generation, debugging, documentation, design assistance, learning concepts)
- Approximate extent of usage
- Examples of tasks where AI assistance was used
- How generated content was reviewed, modified, or validated by the team

The report should clearly distinguish between work produced by team members and assistance provided by AI tools. Failure to disclose significant AI usage may be considered an academic integrity violation.

7. Suggested Project Roadmap & Timeline

Phase 0 — Setup & Planning

- Set up GitHub repo, branching strategy, and project board
- Agree on tech stack versions: Java, JavaFX, PostgreSQL, Gson, JDBC
- Design the JSON communication protocol (request type, payload, response codes)
- Draft initial database ERD
- Set up local PostgreSQL instance + DDL scripts
- Create skeleton server (socket listener + thread pool) and skeleton JavaFX client

Deliverable: Server accepts a connection, client sends a "ping", server responds. Empty DB schema created.

Phase 1 — Core Backend & Authentication

- User Registration (unique username/email, hashed password)
- User Login / Logout, session management
- Password validation rules
- Basic profile creation (display name, bio placeholder)
- JavaFX: Login screen, Registration screen, basic landing screen

Deliverable: Users can register, log in, log out, and multiple clients can connect simultaneously.

Phase 2 — Tweets & Feed

- Create Tweet (text only), View Tweet, Delete Tweet
- Timestamp tracking
- User Tweet History
- Chronological feed (basic timeline)
- Real-time tweet delivery to connected followers
- JavaFX: Compose tweet UI, tweet card component, feed list view

Deliverable: Users can post tweets and see them in a feed.

Phase 3 — Social Graph & Interactions

- Follow / Unfollow
- Followers / Following lists + counts
- Personalized feed
- Likes / Unlike
- Replies (basic thread view)

- Retweets (simple version)

Deliverable: Core social network + interactions fully working.

Phase 4 — Final Features, Polish & Documentation

- Hashtags (basic detection + search)
- Image attachments (basic support)
- Search (users + tweets basic)
- UI polish + bug fixing
- Multi-client testing (concurrency issues)
- README + ERD + SQL scripts
- Generative AI usage report

Deliverable: Fully working, documented, submission-ready project.

Bonus (if time remains)

- Dark mode / theme customization
- Character limit enforcement
- Manual feed refresh
- User blocking / muting
- Pinned tweets
- Polls
- GIF / video support
- Real-time notifications and interaction updates (likes, follows, notifications)
- Scheduled tweets / edit history

Notes

- Dependency flow: Auth → Tweets → Social Graph → Feed → Interactions → Hashtags/Media → Search
- Backend + frontend should be developed in parallel in every phase
- Documentation should be updated continuously, not at the end

8. Useful Tools & Packages

You can find a selection of helpful libraries, apps, and tools below. You are not required to use all of them, but they may be useful during development.

- **UUID:** The Universally Unique Identifier class can be used to generate unique IDs for objects that need to be uniquely identified. Note that using UUID for every class is not always necessary or optimal.
 - [What is a UUID?](#)
 - [UUID in Java](#)
- **Scene Builder:** A visual layout tool for designing JavaFX user interfaces using drag-and-drop components. It generates FXML markup that can be integrated into Java applications.
 - [Scene Builder installation tutorial](#)
- **JavaFX:** A Java library for building modern graphical user interfaces. It supports CSS styling, FXML-based UI design, and integrates well with Java for responsive desktop applications.
 - [JavaFX Learning Playlist](#)
- **PostgreSQL:** A powerful open-source relational database system known for reliability, scalability, and strong data integrity.
 - [SQL Tutorial](#)
- **Gson:** A Java library used to convert Java objects into JSON format and vice versa. It supports serialization and deserialization for communication between client and server.
 - [Gson Object Serialization](#)
- **Hashing Libraries (BCrypt):** Used for secure password hashing to avoid storing plaintext passwords.
 - [Hashing a Password in Java](#)

9. Evaluation Criteria

Code Requirements

- Code must compile and run without errors
- Well-structured and readable code
- Proper use of OOP concepts (encapsulation, inheritance, polymorphism, abstraction)
- Git version control with meaningful commits and multiple branches
- Preparedness to present and explain code during evaluation
- Well-written **README.md** for clear project understanding

Evaluation Breakdown

Functionality

- All required features fully implemented
- Application runs correctly without crashes
- Core logic for:
 - User management
 - Twitting and social features
- Proper data storage and retrieval using PostgreSQL

Code Quality & OOP

- Proper application of OOP principles
- Clean, maintainable, and readable code structure
- Consistent Java naming conventions
- Use of comments for complex logic

JavaFX GUI

- User-friendly and intuitive interface
- Responsive UI interactions
- Proper layout and clear information display

Database Design & Integration

- Efficient and logical database schema
- Secure and correct JDBC usage
- Proper data handling between app and database

Version Control & README

- Consistent Git usage with meaningful commits
- Clean branching strategy
- Complete **README.md**

Bonus Features

- Additional features beyond requirements
- Evaluation based on:
 - Complexity, implementation quality, and practical value

10. Submission Requirements

Each team must submit the following:

GitHub Repository

- Complete source code
- Full commit history showing contributions from all team members
- Final **develop** branch merged into **main**

README Documentation

- Project overview
- Setup and installation instructions
- Feature descriptions
- Usage guide

Project Report

- System architecture overview
- Database design (ERD and SQL DDL scripts)
- UML diagrams and other relevant design artifacts
- AI usage disclosure (tools/models used and their role in the project)

Notes

- Keep the repository **private** and grant access only to your teammates and mentor.
- Ensure all submitted materials are complete and up to date.

The final project should demonstrate solid software engineering practices and result in a fully functional system suitable for portfolio presentation and technical interviews.

Submission deadlines and evaluation details will be announced separately.